Fondamenti di Analisi dei Dati

from data analysis to predictive techniques

Prof. Antonino Furnari (antonino.furnari@unict.it)

Corso di Studi in Informatica

Dip. di Matematica e Informatica

Università di Catania

Università di Catania

Dimensionality Reduction
Principal Component Analysis (PCA)

Transform high-dimensional data into meaningful, compact representations while preserving essential information.

The Challenge of High-Dimensional Data

Real-World Scale

Modern datasets present unprecedented dimensionality challenges:

- An 800×600 pixel color image: **1,440,000 features**
- Text "bag of words" models: **50,000+ features**
- Every pixel or word is necessary—we can't just delete them

Two Critical Problems

1. The Curse of Dimensionality

In high-dimensional space, everything becomes "far apart," causing distance-based models like KNN to fail.

2. Multicollinearity

Features are highly redundant—adjacent pixels in images contain overlapping information.

Feature Selection vs Feature Reduction

Feature Selection

Choose a subset of original variables by examining correlations, p-values, or using Ridge/Lasso regression.

Risk: Discarding variables throws away information unless they're perfectly reconstructible.

Feature Reduction

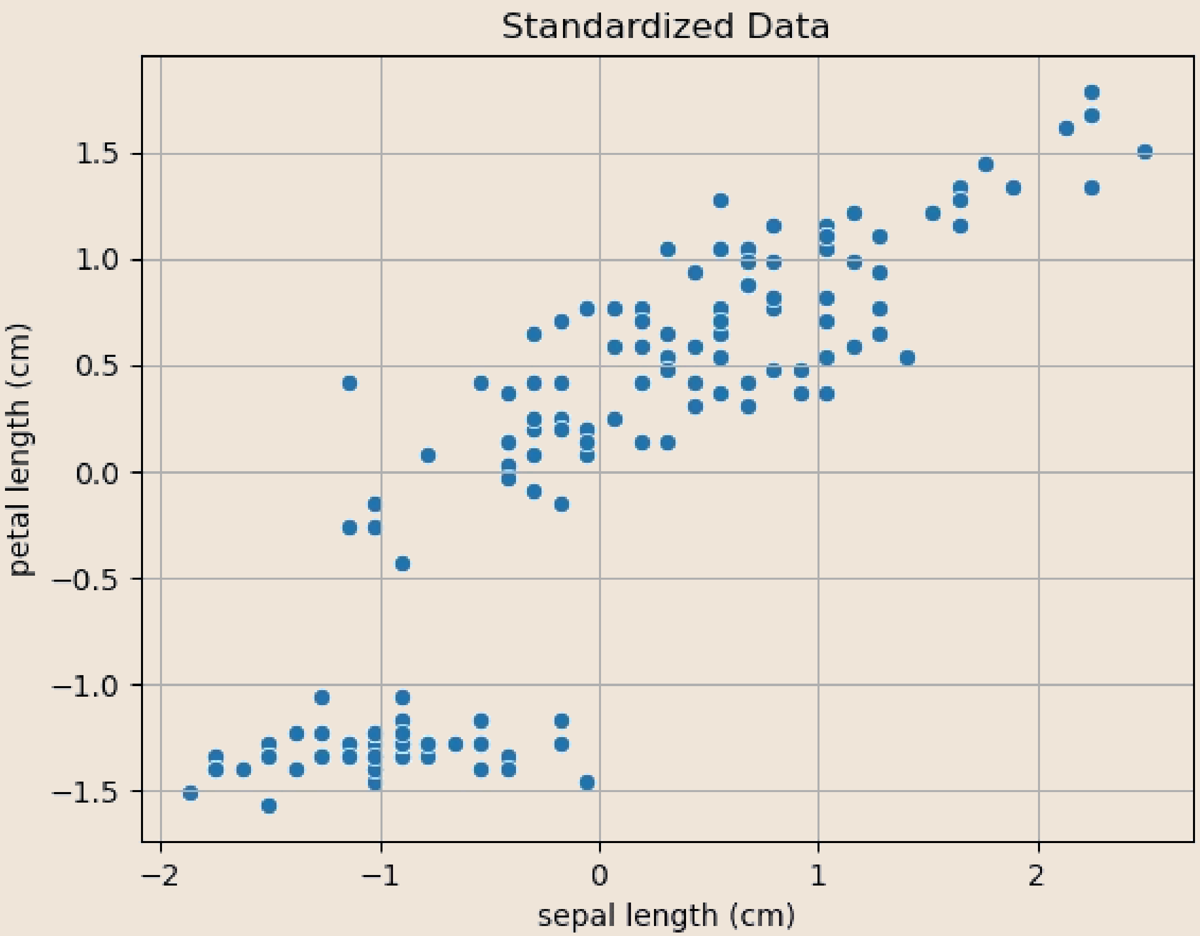
Create a *new set* of features that summarize the original features while losing minimal information.

Transform data from high dimensions to smart, compressed representations.

When features are highly correlated, feature selection discards redundant variables. But what if every feature matters?

That's where **dimensionality reduction** becomes essential—we compress rather than discard.

A Simple Example: The Iris Dataset



Sepal length and petal length are highly correlated (Pearson coefficient: **0.87**, p-value: **1.04e-47**). Instead of choosing one feature, we can project both onto a single dimension that captures maximum variance.

Mapping the 2D data to a single dimension with a linear transformation

2D data can be mapped to a single dimension with a linear transformation as follows:

$$z = u_1 x_1 + u_2 x_2 = \mathbf{u}^T \mathbf{x}$$

The expression above maps point (x_1, x_2) onto a line which passes through the origin (there is no bias term) and has the same direction as $\mathbf{u} = (u_1, u_2)$.

In practice, we often assume $\mathbf{u}$ as a unit vector:

$$||\mathbf{u}||_2 = 1$$

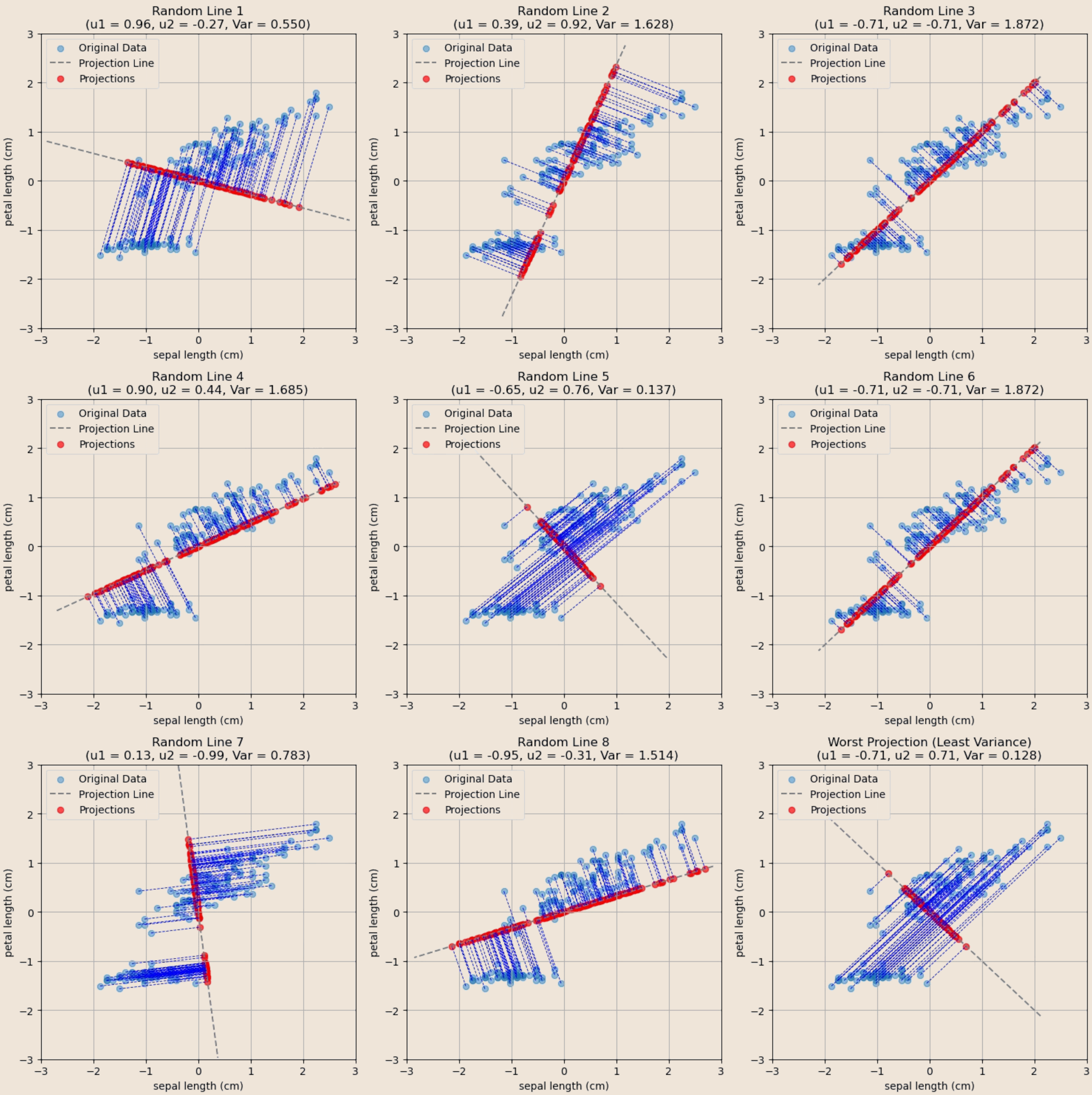
We will assume data is standardised so that axes with higher variance do not dominate.

Finding the Optimal Mapping

Different mappings yield different results. Some compress points together, losing distinctiveness. Others maintain separation.

We quantify distinctiveness through **variance of mapped data**—higher variance means points retain their unique characteristics (two distinct points do not collapse to a single point once mapped).

> The best mapping maximizes variance, ensuring data points maintain their distinctiveness in the reduced space.



General Mathematical Formulation

We have seen a simple example in the case of two variables. Let us now discuss the general formulation in the case of D-dimensional variables.

Let $\{\mathbf{x}_n\}$ be a set of N observations, where $n = 1, \ldots, N$, and $\mathbf{x}_n$ is a vector of dimensionality D. We will define Principal Component Analysis as a projection of the data to $M < D$ dimensions such that the variance of the projected data is maximized.

⚠ NOTE: To avoid any influence by the individual variances along the original axes, we will assume that **the data is standardized**.

Special case: $M = 1$

We will first consider the case in which $M = 1$. In this case, we want to find a D-dimensional vector $\mathbf{u}_1$ such that, projecting the data to this vector, variance is maximized. Without loss of generality, we will choose $\mathbf{u}_1$ such that $\mathbf{u}_1^T \mathbf{u}_1 = 1$. This corresponds to choosing $\mathbf{u}_1$ as a unit vector, which makes sense, as we are interested in the direction of the projection.

Original Data

Let's define some properties of the original data:

Mean

$$\bar{\mathbf{x}} = \frac{1}{N} \sum_{n=1}^{N} \mathbf{x}_n$$

Covariance Matrix

$$\mathbf{S} = \frac{1}{N} \sum_{n=1}^{N} (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^T$$

Projected Data

We know that projecting a data point to $\mathbf{u}_1$ is done simply by the dot product:

$$\mathbf{u}_1^T \mathbf{x}$$

Mean of the projected data

$$\mathbf{u}_1^T \bar{\mathbf{x}}$$

Variance of the projected data

$$\frac{1}{N} \sum_{n=1}^{N} (\mathbf{u}_1^T \mathbf{x}_n - \mathbf{u}_1^T \bar{\mathbf{x}})^2 = \mathbf{u}_1^T \left[\frac{1}{N} \sum_{n=1}^{N} (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^T \right] \mathbf{u}_1 = \mathbf{u}_1^T \mathbf{S} \mathbf{u}_1$$

Maximizing Variance: An Eigenvalue Problem

Our objective is to find the optimal projection vector $\mathbf{u}_1$ that maximizes the variance of the projected data. This can be formalized as an optimization problem:

$$\arg\max\{\mathbf{u}_1^T\mathbf{S}\mathbf{u}_1\} \text{ subject to } \mathbf{u}_1^T\mathbf{u}_1 = 1$$

> The constraint $\mathbf{u}_1^T\mathbf{u}_1 = 1$ is crucial, ensuring we find a direction (a unit vector) rather than arbitrarily scaling the variance. Without it, we could endlessly inflate the variance by choosing vectors with larger magnitudes.

While we won't see the mathematical details, this optimization problem's solution directly leads to the concept of eigenvalues and eigenvectors, where the variance is maximized when:

$$\mathbf{u}_1^T\mathbf{S}\mathbf{u}_1 = \lambda_1$$

Or equivalently:

$$\mathbf{S}\mathbf{u}_1 = \lambda_1\mathbf{u}_1$$

Here, $\mathbf{u}_1$ is an eigenvector of the covariance matrix $\mathbf{S}$, and λ_1 is its corresponding eigenvalue, representing the variance captured along that direction.

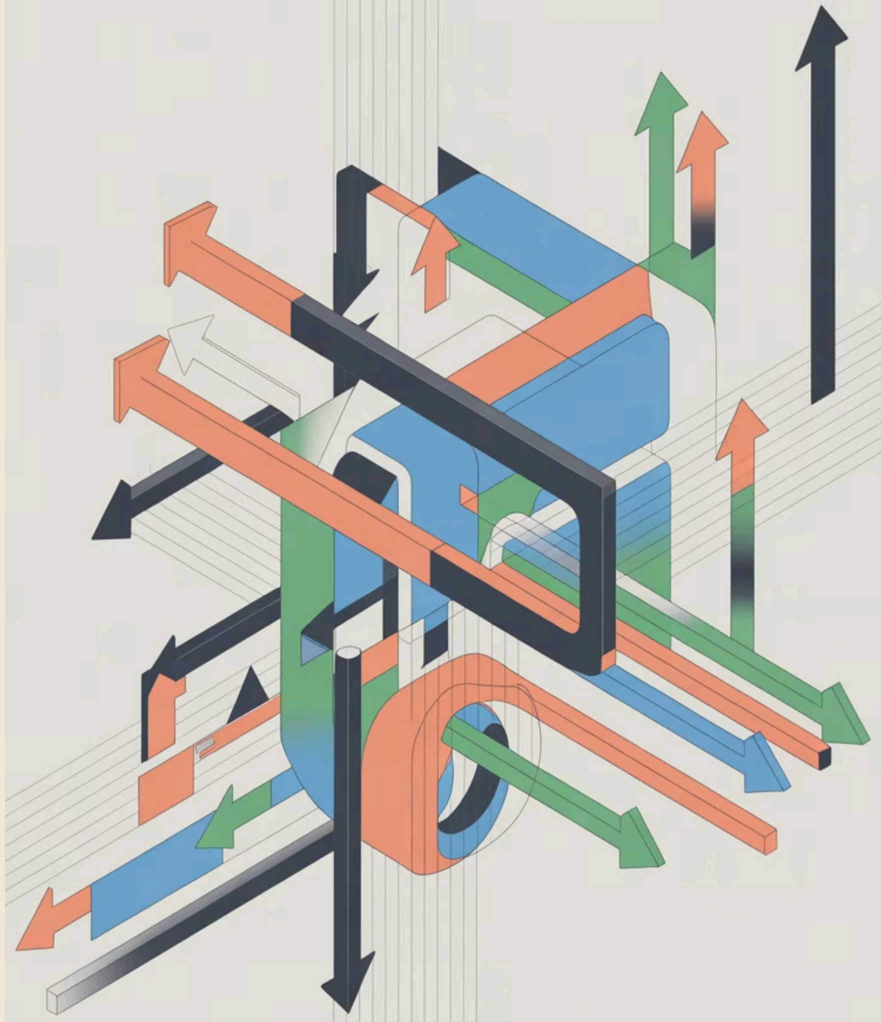
Since we want to maximize $\mathbf{u}_1^T\mathbf{S}\mathbf{u}_1 = \lambda_1$, in practice, we will choose as $\mathbf{u}_1$the **eigenvector corresponding to the largest eigenvalue.**

The Elegant Solution: Eigenvectors

The vector $\mathbf{u}_1$ that maximizes the variance of the projected data is defined as the **first principal component** of the data. This vector corresponds to the eigenvector of the covariance matrix $\mathbf{S}$ with the largest eigenvalue λ_1.

We can proceed in an iterative fashion to find the other components. To construct a new, orthogonal reference system, we select the next principal component, $\mathbf{u}_2$, such that it is orthogonal to the first principal component, $\mathbf{u}_1$ ($\mathbf{u}_2 \perp \mathbf{u}_1$). The second principal component is then the vector $\mathbf{u}_2$ that maximizes the projected data's variance **among all vectors orthogonal to $\mathbf{u}_1$.**

In practice, it can be shown that the M principal components are found by choosing the M eigenvectors $\mathbf{u}_1, \ldots, \mathbf{u}_M$ of the covariance matrix $\mathbf{S}$ that correspond to the M largest eigenvalues $\lambda_1, \ldots, \lambda_M$.



Matrix Formulation

To see PCA in matrix form, we define the data matrix $\mathbf{X}$ as:

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \dots \\ \mathbf{x}_N \end{pmatrix} = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{1D} \\ x_{21} & x_{22} & \dots & x_{2D} \\ \dots & & & \\ x_{N1} & x_{N2} & \dots & x_{ND} \end{pmatrix}$$

$\mathbf{X}$ is the $[N \times D]$ data matrix, with N observations and D original features.

We also define the transformation matrix $\mathbf{W}$ as:

$$\mathbf{W} = \begin{pmatrix} \mathbf{u}_1 \\ \mathbf{u}_2 \\ \dots \\ \mathbf{u}_M \end{pmatrix} = \begin{pmatrix} u_{11} & u_{12} & \dots & u_{1D} \\ u_{21} & u_{22} & \dots & u_{2D} \\ \dots & & & \\ u_{M1} & u_{M2} & \dots & u_{MD} \end{pmatrix}$$

$\mathbf{W}$ is an $[M \times D]$ matrix, where each row is a principal component vector $\mathbf{u}_i$ of the covariance matrix $\mathbf{S}$.

We can transform the entire data matrix with PCA as follows:

$$\mathbf{Z} = \mathbf{X}\mathbf{W}^T$$

The projected data, $\mathbf{Z}$, is an $[N \times M]$ matrix. Each row represents an observation in the reduced M-dimensional PCA space. We call these new dimensions **latent variables** as they uncover underlying data patterns and their values are not directly observed.

A Computational Note

In practice, we typically compute all D eigenvectors of the covariance matrix $\mathbf{S}$ and arrange them into a full transformation matrix $\mathbf{W}_{\text{all}}$, sorted by their corresponding eigenvalues in descending order:

$$\mathbf{W}_{\text{all}} = \begin{pmatrix} \mathbf{u}_1 \\ \mathbf{u}_2 \\ \dots \\ \mathbf{u}_D \end{pmatrix}$$

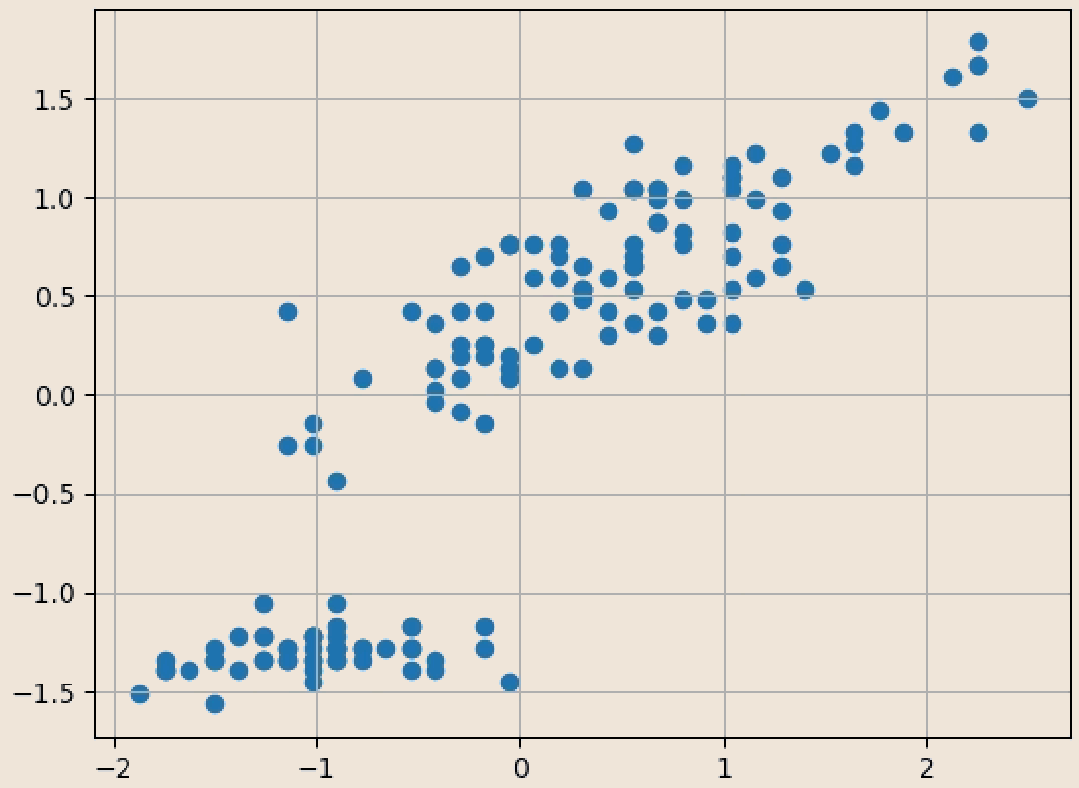
We then "truncate" $\mathbf{W}_{\text{all}}$ to select the top M eigenvectors, which form our desired transformation matrix $\mathbf{W}$. This matrix $\mathbf{W}$ contains the M principal components.

For instance, to reduce to $M = 2$ dimensions, we would select the two eigenvectors corresponding to the largest eigenvalues:

$$\mathbf{W} = \begin{pmatrix} \mathbf{u}_1 \\ \mathbf{u}_2 \end{pmatrix}$$

This approach simplifies the process, allowing for efficient computation of the principal components.

Back to Our Iris Example



Covariance Matrix

```
[[1.007  0.878]
 [0.878  1.007]]
```

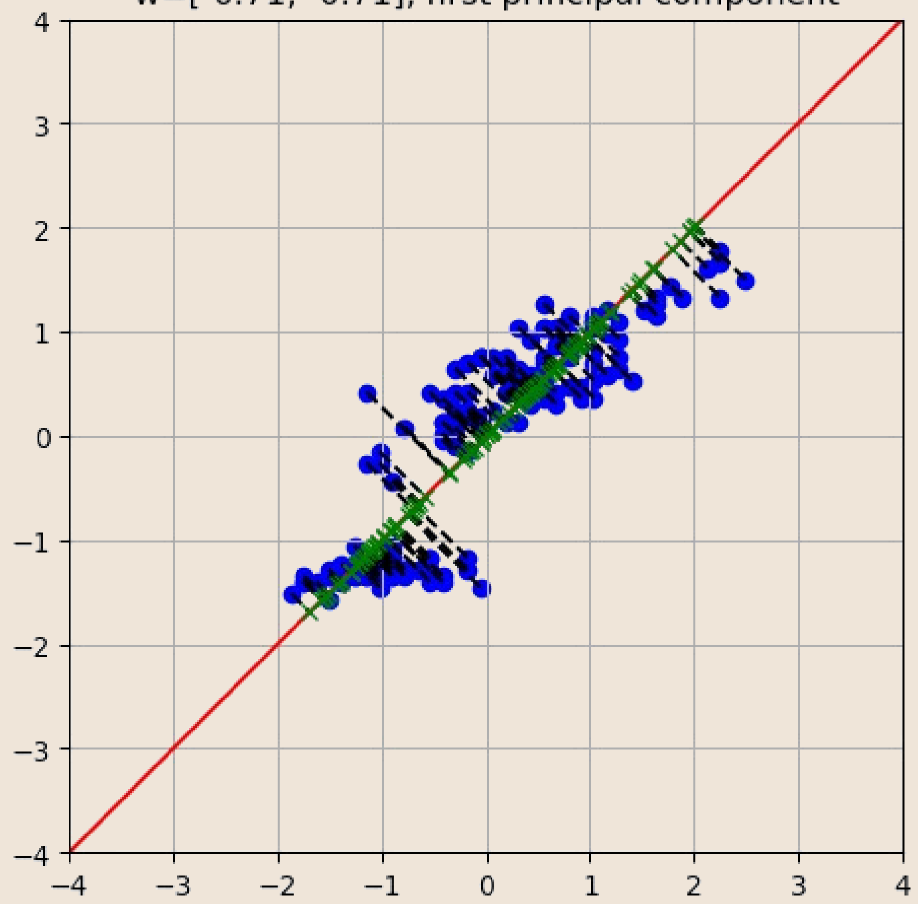
High correlation (0.878) confirms redundancy between features.

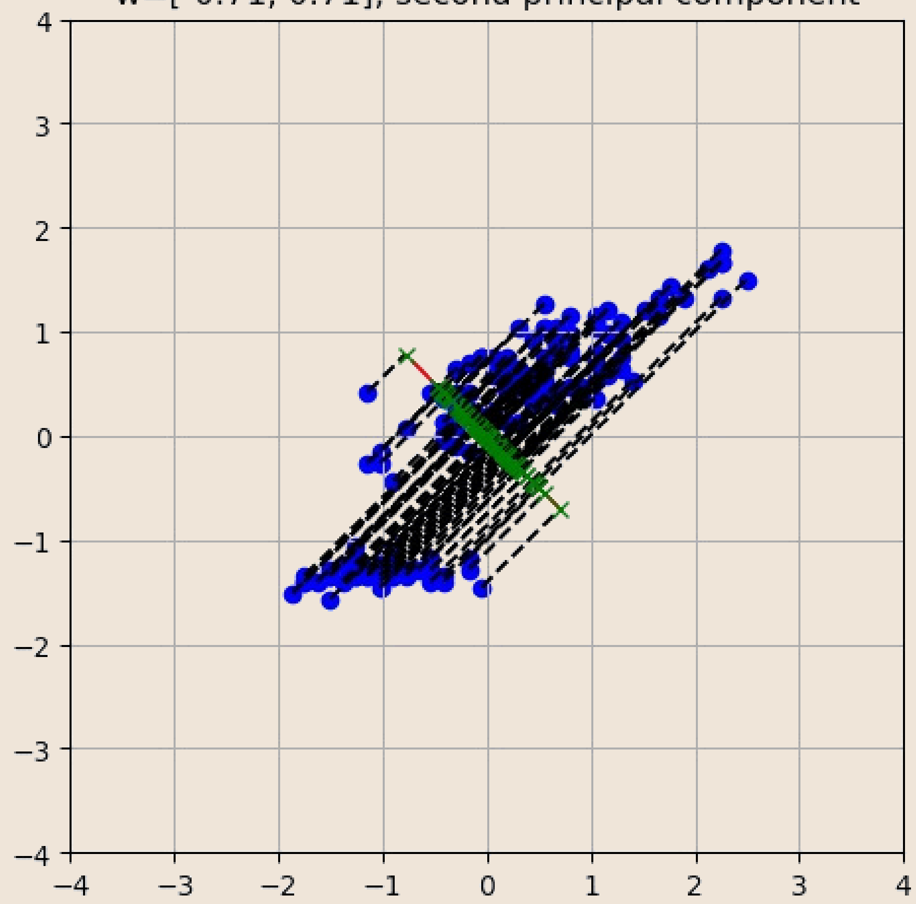
Eigenvalues & Eigenvectors

```
λ: [1.884, 0.129]
u: [[-0.707, -0.707]
   [-0.707,  0.707]]
```

Eigenvectors sorted by eigenvalues are the two principal components.

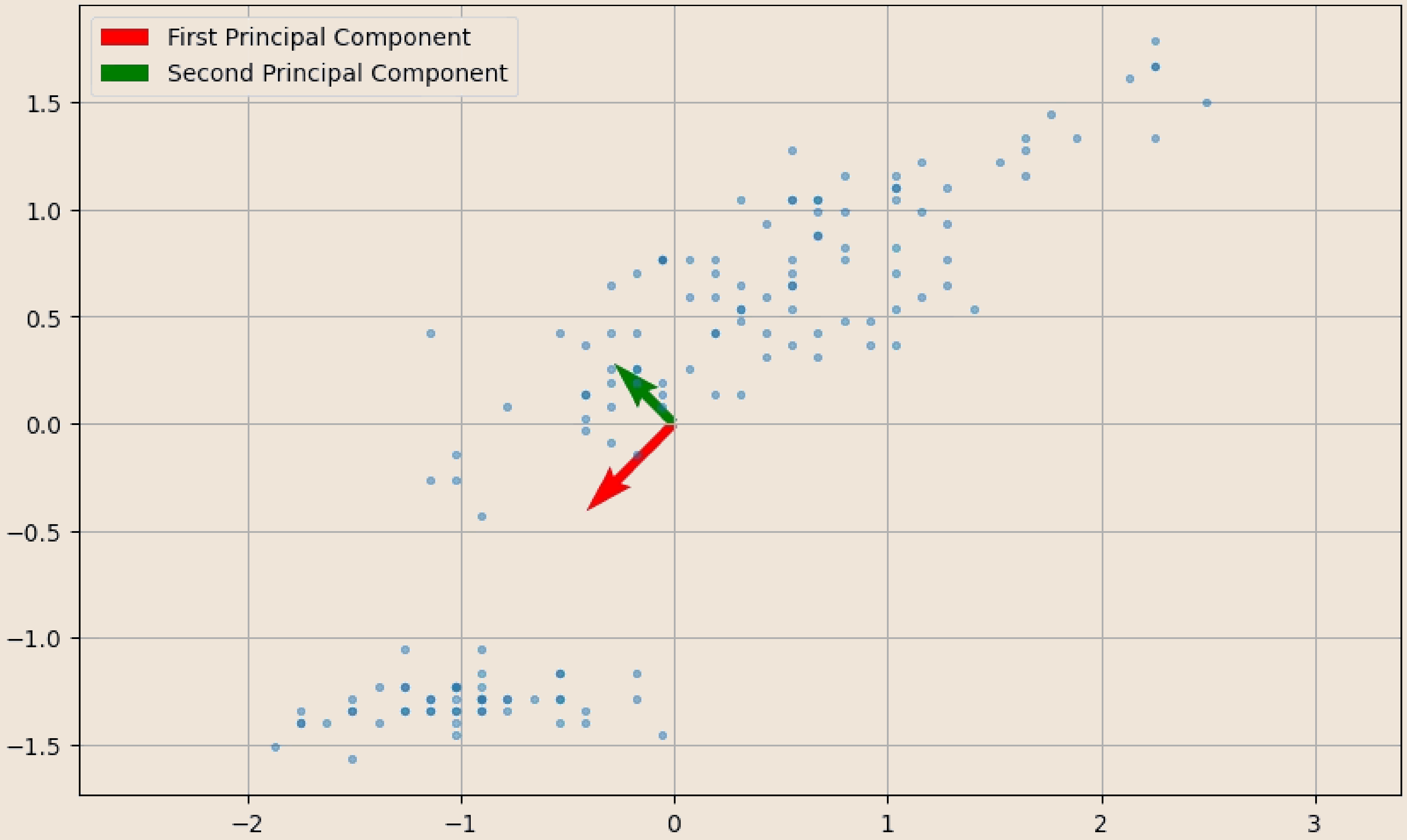
Mapping of points to the lines identified by the two principal components





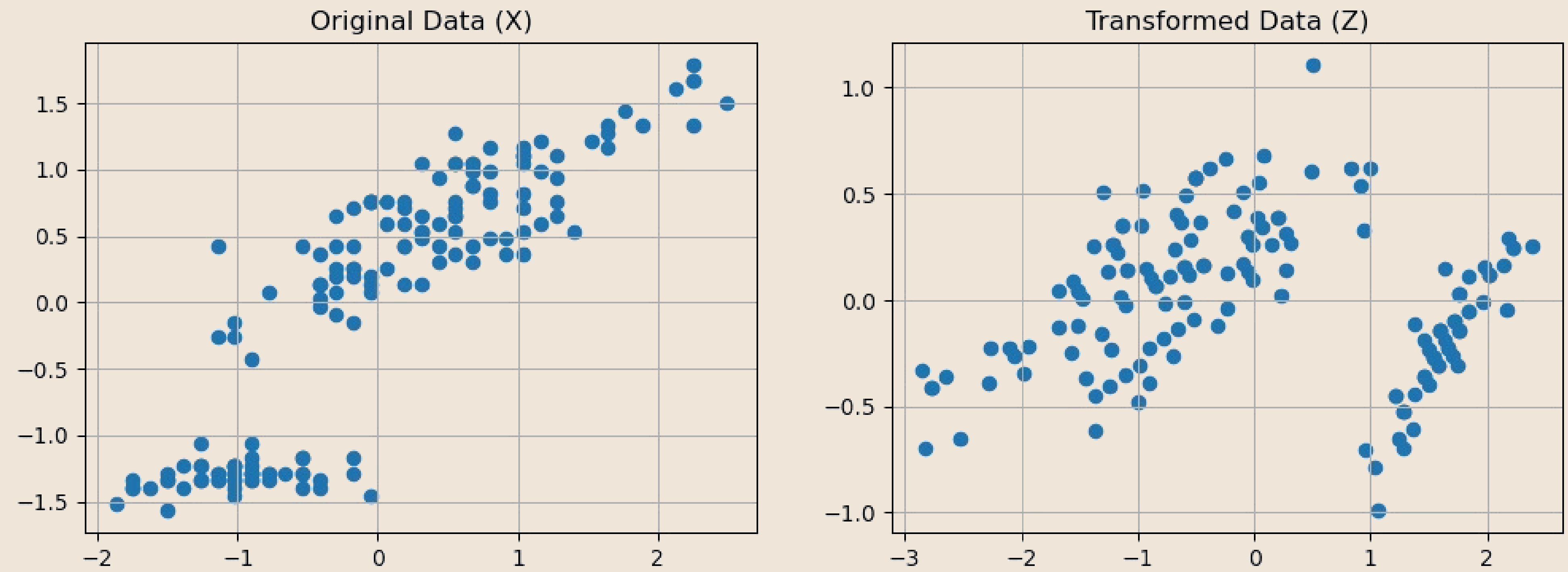
Visualizing Principal Components

The two principal components identify orthogonal directions of maximum variation. The first component (longer arrow) captures most variance; the second captures remaining variation perpendicular to the first one.



PCA as Data Rotation

If we set $M = D$, we can see PCA as data rotation around the origin (recall that the data is standardised).



When $M < D$ we can see PCA as a two-step process:

1

Step 1: Rotate

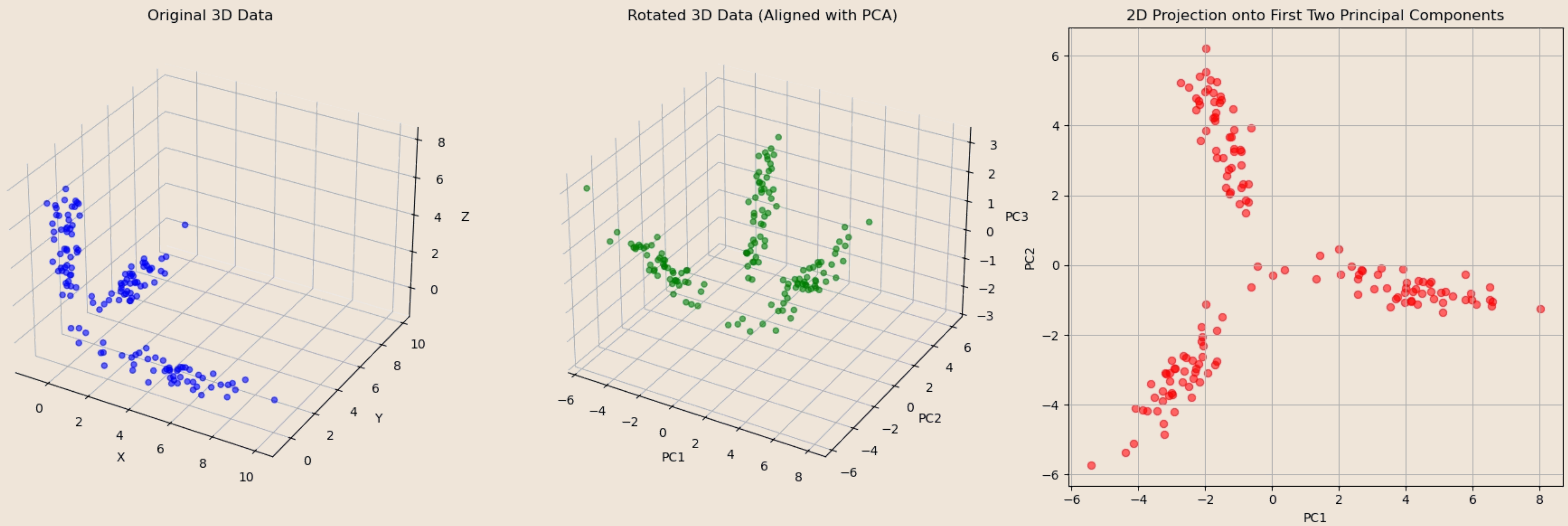
Transform data so first principal component becomes the x-axis

2

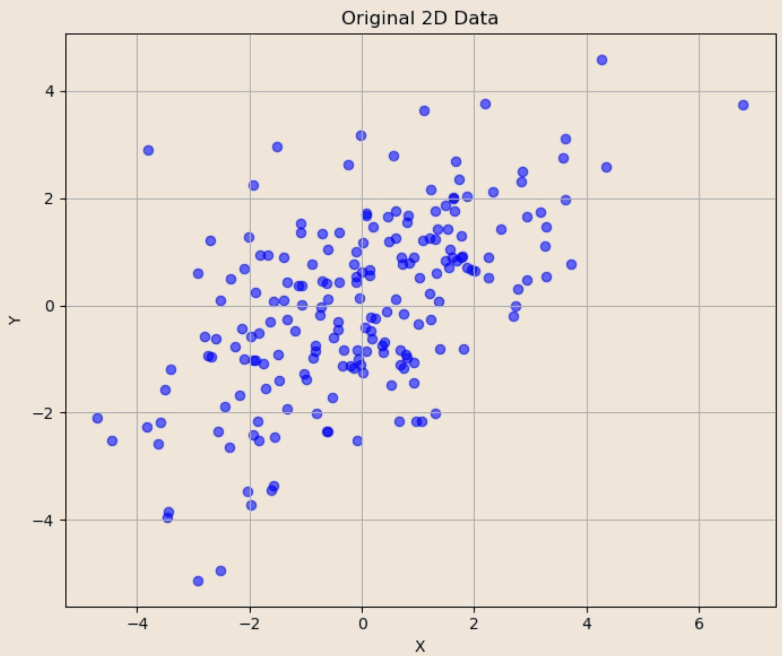
Step 2: Project

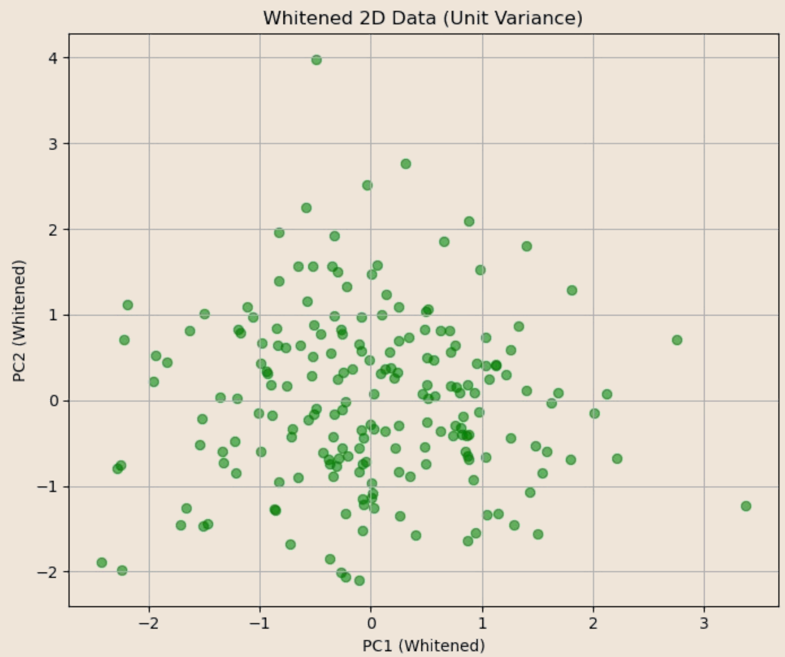
Drop the higher dimensions to reduce from D-dimensional to M-dimensionale

For instance, in 3D:



PCA for Data Whitening





Decorrelation Property

PCA transforms data with covariance matrix ∑ into new variables with **diagonal covariance matrix ∑'**. This means new variables are completely uncorrelated.

Why it works: We rotated data so axes align with main variation directions and constructed components to be orthogonal.

Original Covariance

```
(array([[3.60543087, 1.84308349],
       [1.84308349, 2.83596216]]),
```

Covariance of the Transformed Data

```
 array([[ 1.00000000e+00, -1.40945288e-16],
       [-1.40945288e-16,  1.00000000e+00]]))
```

> **Application:** Whitening is valuable for linear regression with correlated variables—preprocessing with PCA removes multicollinearity and stabilises fitting, though interpretability decreases.

Choosing the Number of Components

How many components M should we keep? The answer depends on how much information we need to retain. We measure this through **cumulative explained variance**.

Let's consider the whole Fisher Iris Dataset (4 features).

Principal Components

components	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
z1	0.361387	0.656589	-0.582030	0.315487
z2	-0.084523	0.730161	0.597911	-0.319723
z3	0.856671	-0.173373	0.076236	-0.479839
z4	0.358289	-0.075481	0.545831	0.753657

Covariance Matrix of the Transformed Data

	z1	z2	z3	z4
z1	4.23	0.00	0.00	0.00
z2	0.00	0.24	-0.00	0.00
z3	0.00	-0.00	0.08	-0.00
z4	0.00	0.00	-0.00	0.02

Let

$$Var(z_i)$$

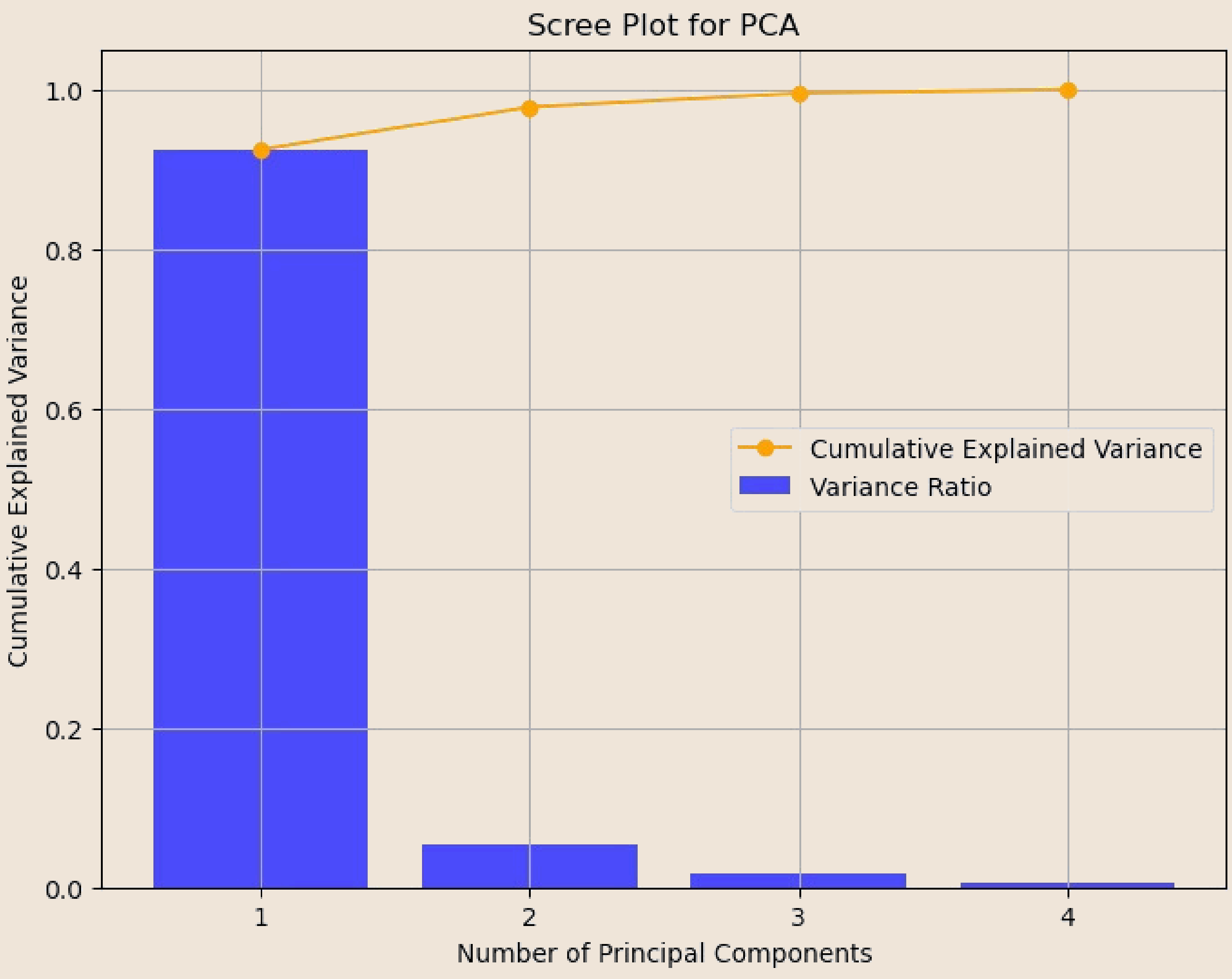
variance of the data along the z_i component. The total variance is given by the sum of the variances:

$$V = var(z_1) + var(z_2) + \ldots + var(z_D)$$

We can quantify the fraction of variance explained by each component n as follows:

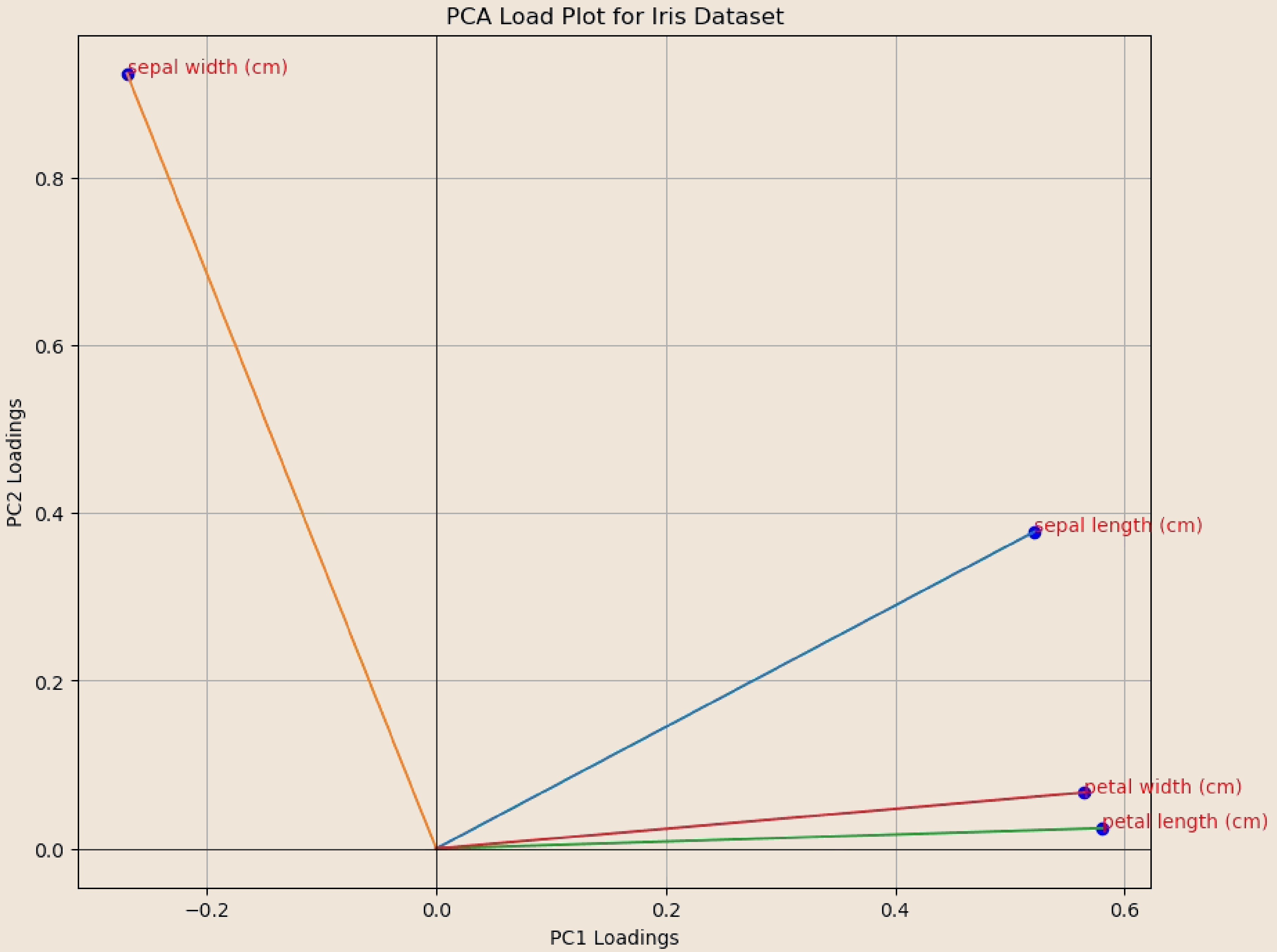
$$\frac{var(z_n)}{V(D)}$$

In practice, it is common to visualize the explained variance ratio in a **scree plot**:



Interpreting Components: Load Plots

Load plots visualise original variables based on the values of the first two components, which are often called "loadings". Loading with large absolute values are more influential in determining the value of the principal components.



Few rules to interpret and use a load plot:

Clustered Variables

Variables appearing together are positively correlated; opposite sides indicate negative correlation

Distance from Origin

Variables farther from center have higher loadings—they're more influential in computing principal components

Feature Selection Guide

Identify correlated variable groups; select one representative from each cluster to reduce redundancy

Laboratory: Applications of PCA

Dimensionality Reduction & Visualization

Simplify complex datasets by reducing the number of features while retaining most information, enabling clearer visualizations of high-dimensional data trends.

Principal Component Regression (PCR)

Overcome multicollinearity issues in linear regression models by using uncorrelated principal components as predictors, improving model stability and interpretability.

Data Compression

Reduce storage requirements and speed up data transmission by representing data with fewer principal components, preserving essential variance and patterns.

Eigenfaces

A classic computer vision application for facial recognition, where faces are represented as combinations of principal components extracted from a set of face images.

Conclusions

We Have Explored:

- The challenge of high-dimensional data
- Feature selection vs feature reduction
- Principal Component Analysis (PCA) fundamentals
- Mathematical formulation and eigenvalue problems
- Matrix formulation of PCA
- Choosing the number of components
- Applications: dimensionality reduction, visualization, PCR, data compression, eigenfaces

References

- Parts of Chapter 12 of [1]

[1] Bishop, Christopher M., and Nasser M. Nasrabadi. Pattern recognition and machine learning. Vol. 4. No. 4. New York: springer, 2006.